

Appunti di Analisi e Progettazione del Software

Nicholas Scorrano

A.A 2025/2026

Capitolo 3

Diagrammi SSD e Contratti

3.1 Sequence State Diagram

3.1.1 Cosa sono gli SSD?

Un diagramma di sequenza di sistema (SSD) è un diagramma UML che mostra gli eventi in input e output dei sistemi in discussione

3.1.2 Quali sono gli obiettivi di un SSD?

L'obiettivo principale della creazione di un SSD è comprendere le interazioni che sono implicate e descritte nei casi d'u

Il diagramma serve per visualizzare in modo immediato le funzioni e il comportamento del sistema con un approccio a "scatola nera", il suo scopo è descrivere che cosa fa il sistema in risposta agli stimoli esterni, omettendo e senza spiegare come lo fa al suo interno

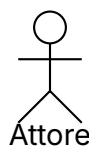
3.1.3 Perché disegnare gli SSD?

Disegnare gli SSD è cruciale perché gli eventi di sistema sono fondamentali per l'architettura del software, il quale deve essere progettato esattamente per gestire tali eventi

Utilizzando la notazione UML, serve a illustrare le interazioni tra gli attori e il sistema.

3.1.4 Componenti degli SSD

Attori Esterni

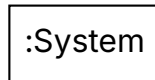


Rappresentano le persone o entità che interagiscono direttamente con il sistema. Il loro ruolo è quello di partecipare allo scenario generando gli eventi (input).

Il loro ruolo è quello di partecipare allo scenario generando gli eventi (input) per richiedere al sistema l'esecuzione di specifiche operazioni.

Per distinguere visivamente un attore umano da un attore che è un sistema informatico, si usa spesso la dicitura «actor» all'interno di un rettangolo per quest'ultimo

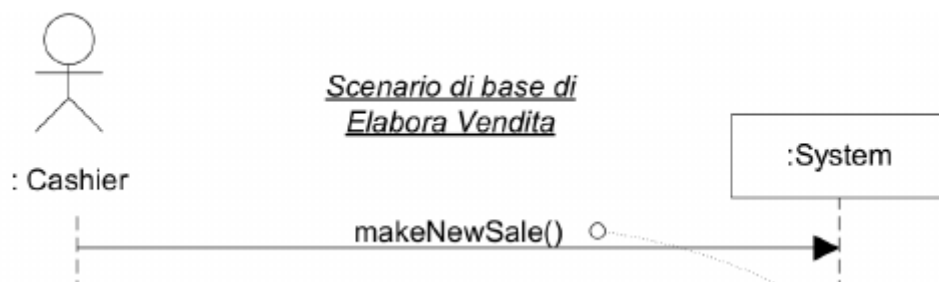
Il Sistema



Viene rappresentato come un'unica entità indivisibile (spesso etichettata graficamente con :System).

La sua funzione nel diagramma è quella di mostrare che cosa fa il sistema ricevente in risposta agli input, ma omettendone volutamente i processi interni su come lo fa (da qui il termine "scatola nera").

Eventi di sistema



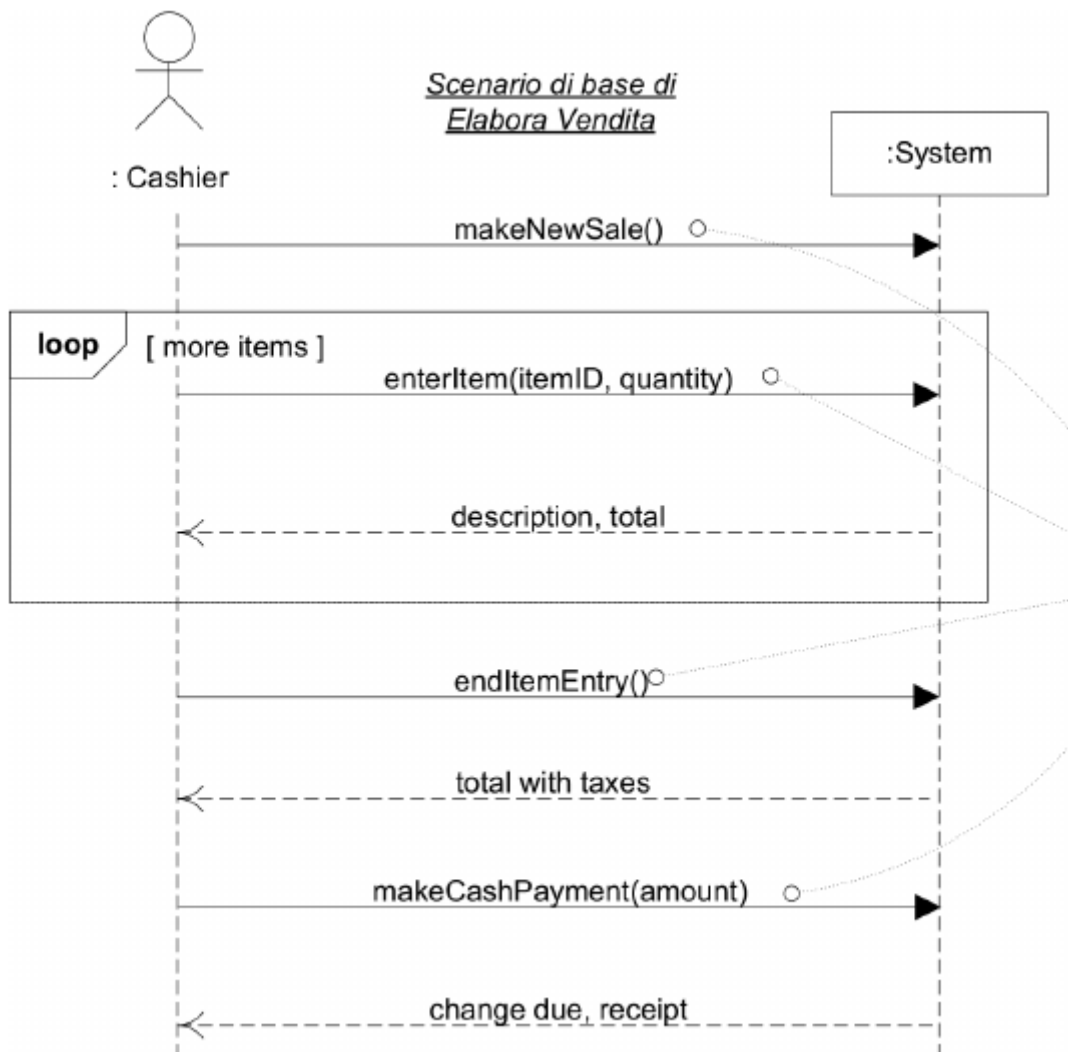
Sono eventi esterni generati dagli attori per richiedere un'operazione di sistema e sono disposti seguendo il loro preciso ordine cronologico.

Sono eventi esterni generati dagli attori e sono disposti seguendo il loro preciso ordine cronologico.

Graficamente appaiono come linee continue a forma di freccia e possono includere l'invio di parametri,

I nomi di questi eventi rappresentano un'intenzione astratta e dovrebbero iniziare con un verbo.

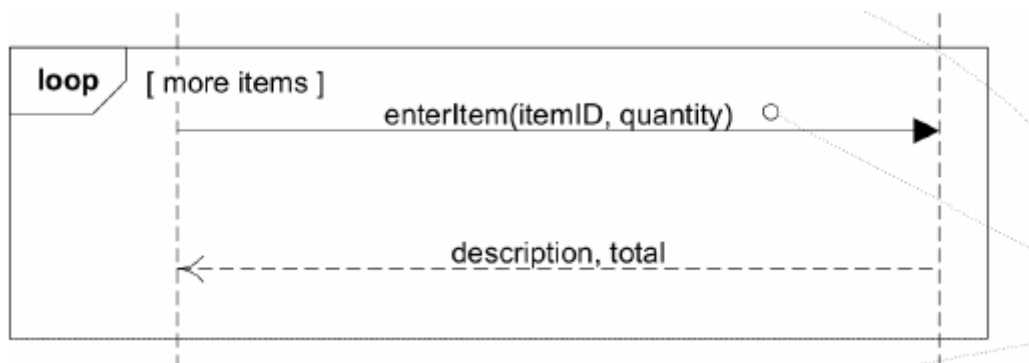
Valori di ritorno



Sono le risposte o le informazioni che il sistema produce e rimanda all'attore in seguito all'elaborazione di un evento.

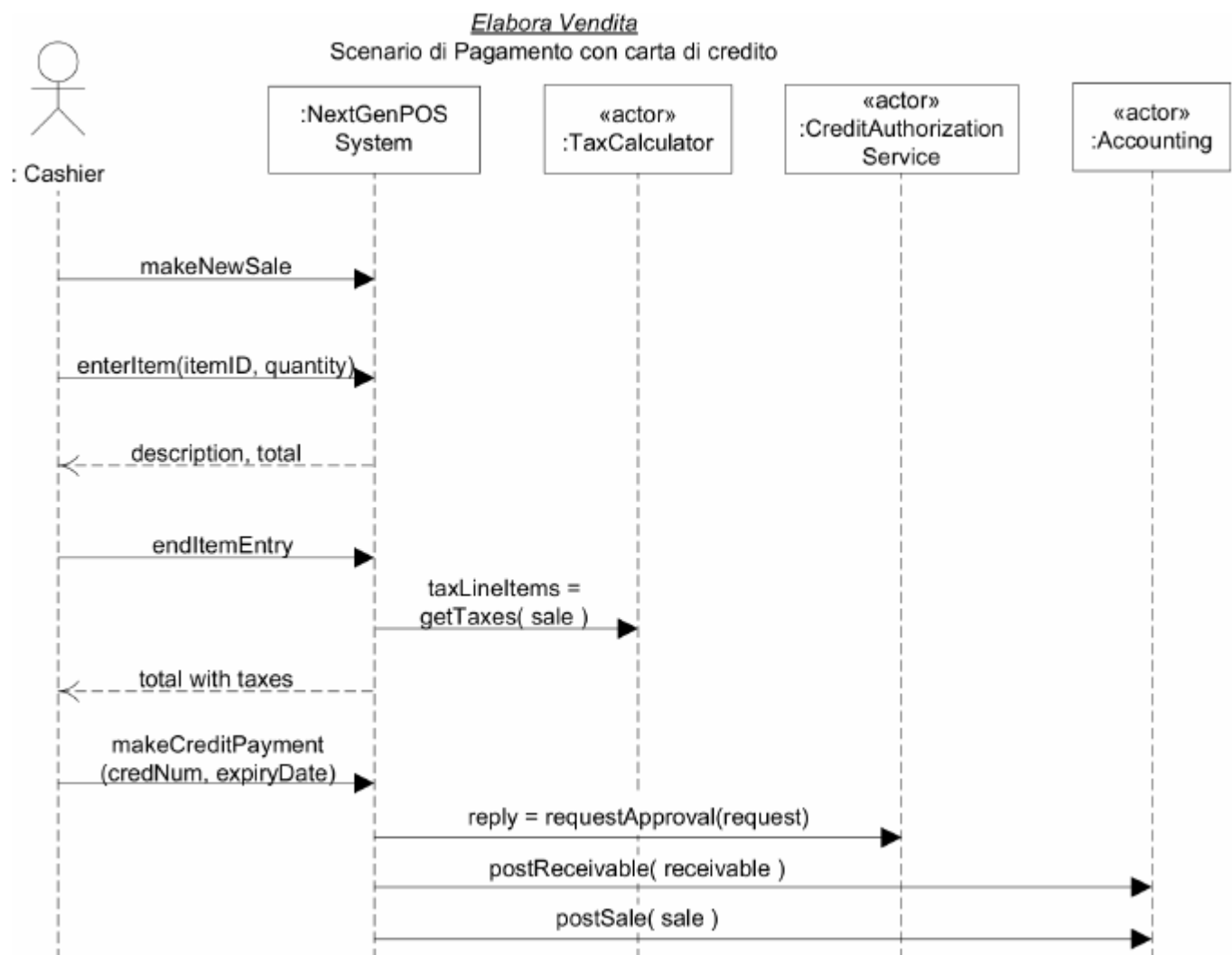
Vengono tracciati utilizzando una linea tratteggiata, si tratta di una componente opzionale che pu essere omessa nel caso in cui l'operazione non restituisca niente

Frame di interazione



Sono costrutti grafici a forma di riquadro utilizzati per raggruppare e strutturare specifiche parti dell'interazione

Sistemi esterni (ed eventi inter - sistema)



Qualora lo scenario preveda che il sistema debba delegare operazioni ad altri enti informatici, il diagramma includerà la rappresentazione di questi attori-sistemi aggiuntivi e mostrerà le interazioni necessarie con essi

3.1.5 Relazione con gli altri elaborati UML

L'SSD non è un elaborato isolato, ma si inserisce all'intero del processo UP in strettissima relazione con gli altri documenti dell'analisi e della progettazione, fungendo da "ponte" tra i requisiti testuali e la progettazione vero e propria

Casi d'Uso

Gli SSD fanno formalmente parte del Modello dei casi d'uso.

Mentre i casi d'uso descrivono il compoeramento e i requisiti in forma testuale, l'SSD prende un particolare scenario e ne visualizza graficamente le interazioni implicate

Contratti delle Operazioni di Sistema

La relazione è di stretta dipendenza procedurale, si parte dall'SSD per poter creare i contratti.

Le frecce di input (eventi di sistema) che vanno dall'attore verso il sistema nell'SSD rappresentano la richiesta di esecuzione di un'operazione di sistema.

Il primissimo passo per scrivere i contratti è proprio "identificare operazioni di sistema dagli SSD"

Una volta individuate, per le operazioni più complesse si scrive un contratto che descrive dettagliatamente come lo stato del sistema cambierà a seguito di quell'evento

Glossario

Poichè un SSD è un diagramma grafico, i nomi delle operazioni, i parametri inviati e i valori restituiti sono scritti in modo estremamente conciso.

Affinchè non ci siano ambiguità durante la successiva fase di progettazione su "cosa entra e cosa esce", la linea guida prevede che tutti i dettagli di questi elementi sintetici dell'SSD vengano spiegati in modo approfondito nel Glossario

Modello di Dominio

L'SSD si collega al Modello di Dominio attraverso gli eventi che genera.

L'esecuzione di un'operazione di sistema ha lo scopo di cambiare lo stato del sistema, e questo stato è rappresentato proprio dagli oggetti presenti nel modello di Dominio.

Inoltre, analizzando gli eventi dell'SSD per scrivere i contratti, è del tutto normale accorgersi di dover aggiornare il Modello di Dominio, registrando nuove classi concettuali, associazioni o attributi che prima non erano emersi

3.2 Contratti delle operazioni di sistema

3.2.1 Cosa sono i Contratti delle Operazioni di Sistema?

I contratti delle operazioni di sistema sono documenti che descrivono il comportamento del software in modo molto più dettagliato rispetto a quanto facciano i casi d'uso.

Mentre un caso d'uso racconta una storia, il contratto si concentra su un singolo evento di input (un'operazione) e definisce con precisione quali cambiamenti quello specifico evento provoca all'interno del sistema (ovvero agli oggetti del modello di dominio)

Tutto questo viene fatto mantenendo un approccio a "scatola nera", il contratto descrive i risultati finali dell'operazione, senza mai spiegare la logica interna usata per ottenerli

Cosa è un operazione di sistema?

Un'operazione di sistema è una trasformazione o un'interrogazione pubblica che il sistema viene chiamato a eseguire per gestire un evento in input generato da un attore esterno

- **È una reazione a un evento:**

Un'operazione di sistema non parte mai da sola.

La sua esecuzione è sempre richiesta da un evento di sistema.

- **Modifica lo stato del sistema:**

Quando viene eseguita, un'operazione cambia lo stato del software.

In fase di analisi, questo cambiamento si traduce graficamente nello stato degli oggetti presenti nel Modello di Dominio.

- **Costituisce l'interfaccia del sistema :**

L'insieme di tutte le operazioni di sistema pubbliche definiscono l'intera interfaccia del software, ovvero l'elenco esatto di "tutto ciò che il sistema sa fare" in risposta agli stimoli esterni

3.2.2 Struttura dei Contratti

Nome Contratto	
Operazione	<i>Nome dell'operazione e parametri</i>
Riferimenti	<i>casi d'uso correlati</i>
Pre-condizioni	<i>Ipotesi sullo stato del sistema prima dell'operazione</i>
Post-condizioni	<i>Cambiamenti di stato dopo l'operazione (scritte al passato)</i>

La struttura di un contratto delle operazioni di sistema si divide in 4 sezioni principali :

1. **Operazione** : Indica il nome e i parametri, ovvero la firma dell'operazione
2. **Riferimenti** : Specifica i casi d'uso all'interno dei quali questa operazione può verificarsi
3. **Pre-Condizioni** : Descrive le ipotesi significative e non banali relative allo stato del sistema, o degli oggetti presenti nel Modello di Dominio, prima che l'operazione venga eseguita.
Questa sezione funge da sintesi dello stato di avanzamento del caso d'uso e serve a comunicare al lettore quali informazioni il sistema deve necessariamente già conoscere o aver soddisfatto
4. **Post-Condizioni** : È considerata la sezione più importante del contratto e descrive i cambiamenti che sono avvenuti nello stato degli oggetti all'interno del Modello di Dominio che l'operazione è stata completata.

Le dichiarazioni in questa sezione non devono descrivere le azioni interne eseguite dal sistema, ma unicamente i risultati ottenuti

Inoltre, devono essere scritte rigorosamente con un verbo al passato e rientrare sempre in una di queste 3 categorie di cambiamento :

1. La creazione o cancellazione di un oggetto
2. La modifica del valore di un attributo
3. La formazione o rottura di un collegamento

3.2.3 Relazione con gli altri elaborati UML

I contratti delle operazioni di sistema hanno una stretta relazione di interdipendenza con gli altri elaborati del processo UP e di UML

Fanno formalmente parte del Modello dei casi d'uso e fungono da ponte tra l'analisi dei requisiti e la progettazione vera e propria del software

Casi D'uso

I contratti sono complementari ai casi d'uso.

Mentre i casi d'uso descrivono il comportamento e i requisiti del sistema in forma testuale e generale, i contratti intervengono per descrivere in modo molto più dettagliato situazioni complesse che non sarebbe opportuno o pratico inserire normalmente nel normale testo del caso d'uso.

Non a caso, la struttura del contratto prevede una sezione "Riferimenti" che indica esplicitamente in quale caso d'uso si verifica l'operazione

Diagrammi SSD

La creazione dei contratti dipende proceduralmente dagli SSD.

Il primo passo operativo per scrivere i contratti consiste proprio nell'identificare le operazioni di sistema guardando gli SSD.

L'SSD mostra l'evento di sistema generato dall'attore, e il contratto entra nel dettaglio di come quell'operazione invocata cambi lo stato del sistema

3.2.4 Modelli di Dominio

Il legame è fortissimo, i contratti utilizzano le pre-condizioni e post-condizioni unicamente per descrivere i cambiamenti agli oggetti presenti all'interno del Modello di Dominio.

3.2.5 OCL - Object Constraint Language

Nell'ecosistema UML, un'operazione ha una firma ed è associata a un insieme di vincoli per specificarne la semantica.

In teoria, i vincoli descritti nel contratto possono essere espressi in modo rigorosamente formale avvalendosi dell'OCL, un linguaggio nato in UML appositamente per questo scopo

Capitolo 4

Dall'Analisi al Design

Il passaggio "Dall'analisi al design" (o alla progettazione) rappresenta un cambio di prospettiva fondamentale nello sviluppo del software

4.1 Analisi ("Fare la cosa giusta")

L'analisi dei requisiti e l'analisi a oggetti servono per capire e definire esattamente cosa deve fare il sistema per soddisfare le esigenze

Questo lavoro di analisi e modellazione concettual, a differenza di quanto si possa pensare, non richiede settimane di lavoro, ma realisticamente si esaurisce in poche ore/giorni all'interno di una iterazione,

4.2 Progettazione / Design ("Fare la cosa bene")

Il lavoro di progettazione sposta l'accento su come implementare tecnicamente e con la giusta architettura la soluzione

Capitolo 5

Architettura Logica

5.1 Cosa è l'Architettura Logica?

L'architettura logica di un sistema software è l'organizzazione su larga scala delle classi software in package (o namespace), sottoinsiemi e strati.

È chiamata architettura logica poichè non vengono prese decisioni su come questi elementi siano distribuiti sui processi o sui diversi computer fisici di una rete

5.2 Definizione di Architettura a Strati

È uno stile di Architettura Logica in cui il sistema è formato da un insieme di elementi chiamati strati.

Ogni strato è un gruppo a grana molto grossa di classi, package o sottosistemi.

5.2.1 Organizzazione degli strati

Gli strati sono organizzati in modo tal che stati "più alti ricorrano ai servizi degli strati "più bassi"

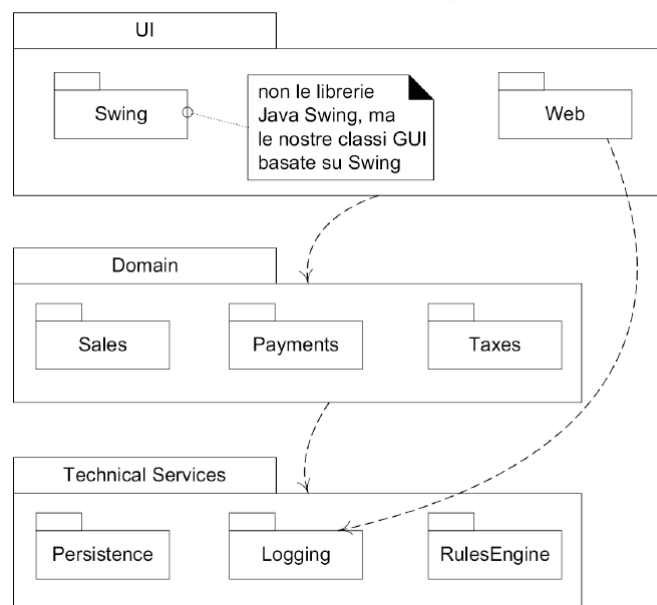


Figura 5.1: Strati mostrati con la notazione di un diagramma dei package di UML

Architettura a strati stretta

Uno strato può solo richiamare i servizi dello stato immediatamente sottostante

Architettura a strati rilassata

Uno strato più alto può richiamare i servizi degli strati più bassi di diversi livelli

5.2.2 Definizione degli strati

Gli strati di un'applicazione software comprendono normalmente i seguenti

User Interface

Oggetti software per gestire l'interazione con l'utente e per la gestione degli eventi

Application Logic e Domain Objects

Oggetti software che rappresentano concetti del dominio (per esempio una classe software *Vendita*) che soddisfano i requisiti dell'applicazione

Technical Services

Oggetti e sottosistemi d'uso generale che forniscono servizi tecnici di supporto (Come l'accesso ai database o il logging degli errori)

5.2.3 Progettazione degli strati

L'obiettivo è organizzare la struttura logica del sistema in strati separati, ciascuno con responsabilità distinte e correlate.

Si attua una separazione netta :

- Gli **strati inferiori** offrono servizi generali e di basso livello
- Gli **strati superiori** sono più specifici per l'applicazione

5.2.4 Vantaggi dell'uso degli strati

L'architettura a strati garantisce una netta separazione degli interessi tra i servizi specifici di alto livello e quelli generali di basso livello

I vantaggi principali includono :

- **Riduzione dell'accoppiamento** e delle dipendenze
- **Aumento della coesione, della chiarezza e del riuso**
- Incapsulazione e decomposizione della complessità
- Possibilità di **sostituire interi strati** con nuove implementazioni senza intaccare il resto del sistema
- Maggiore facilità nello sviluppo da parte di più team, grazie alla netta segmentazione logica
- Possibilità di distribuire alcuni strati su architetture fisiche diverse

5.2.5 Responsabilità coese e separazione degli interessi

All'interno di ciascun strato, le responsabilità degli oggetti devono essere fortemente coese tra loro e non devono mai essere mescolate con le responsabilità di altri strati

5.3 Cosa è un'Architettura Software?

Un'architettura software riguarda l'organizzazione su larga scala del sistema.

Nello specifico essa è definita come :

- L'insieme delle **decisioni significative** sull'organizzazione di un sistema software
- La **scelta degli elementi strutturali** da cui è composto il sistema e delle relative interfacce
- Il **comporamento** specificato dalla collaborazione tra questi elementi
- La **composizione** di questi elementi strutturali e comportamentali in sottosistemi via via più ampi
- Lo **stile architetturale** che fa da guida a questa organizzazione (come ad esempio l'architettura a strati)

5.4 Diagrammi dei Package UML

L'architettura logica di un sistema può essere illustrata mediante un diagramma dei package di UML

5.4.1 Elementi dei diagrammi package

- Uno **strato** può essere modellato come un **package UML**

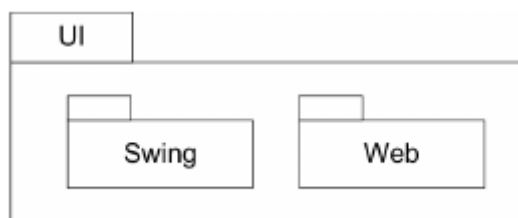


Figura 5.2: Strato di un'architettura rappresentato come package UML

Uno package UML può raggruppare qualunque cosa (classi, altri package, casi d'uso)

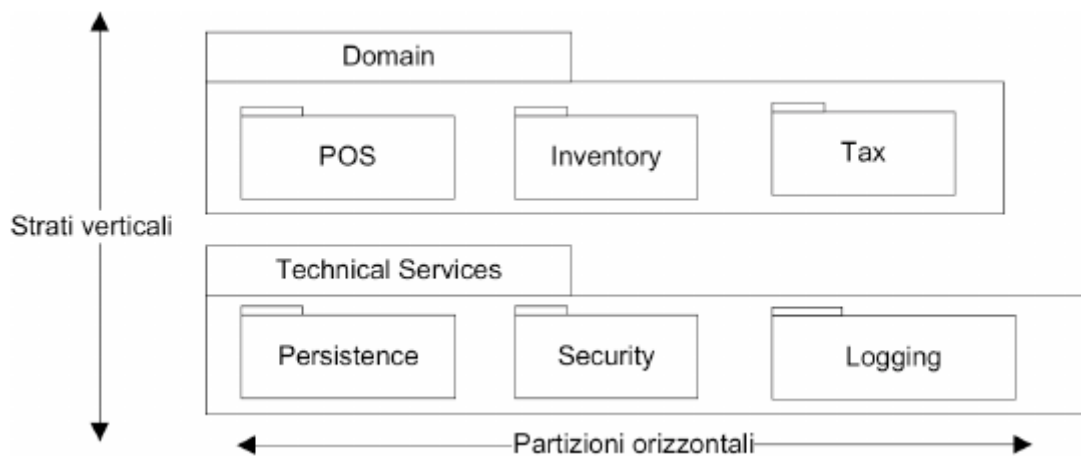
- Una **dipendenza UML** è una linea tratteggiata con una freccia che punta verso il package da cui dipende



Figura 5.3: Esempio di uso di una dipendenza

- Un package UML rappresenta un **namespace** cosicchè è possibile definire due classi con lo stesso nome su package diversi

5.4.2 Definizione : Livelli, Strati e Partizioni



- **Livello (tier)**
Indica solitamente un nodo fisico di elaborazione
- **Strato (layer)**
Rappresenta una sezione verticale dell'architettura logica
- **Partizione (partition)**
È una divisione orizzontale di un sottosistema posizionato all'interno di uno strato

Capitolo 6

Diagrammi di Interazione

6.1 Fondamenti della modellazione orientata agli oggetti

I fondamenti della modellazione orientata agli oggetti si basano sul concetto di **oggetto**. Ogni oggetto nel sistema rappresenta un'istanza di una specifica classe, la quale definisce preventivamente l'insieme comune di caratteristiche condivise da tutte le sue istanze

6.1.1 Elementi di un oggetto

Tutti gli oggetti sono definiti da 3 elementi imprescindibili

Identità

Ogni oggetto possiede un proprio identificativo univoco che lo distingue in modo inequivocabilmente dagli altri

Stato (dati)

È determinato dai valori effettivi dei dati memorizzati negli attributi dell'oggetto in un preciso istante temporale

Comportamento (le operazioni)

Rappresenta l'insieme delle azioni e delle operazioni che l'oggetto è in grado di eseguire

6.1.2 Incapsulamento

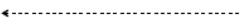
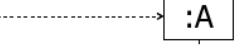
Questa regola stabilisce che i dati devono essere nascosti all'interno dell'oggetto. Di conseguenza, l'unico modo per avvedersi o modificarli è attraverso l'invocazione delle operazioni pubbliche dell'oggetto stesso

6.1.3 Messaggi

Gli oggetti non operano in modo isolato, ma collaborano per eseguire le funzioni del sistema scambiandosi messaggi.

Quando un oggetto invia un messaggio a un altro oggetto, tale messaggio causa l'invocazione della corrispondente operazione nel destinatario

Tipologie di messaggi

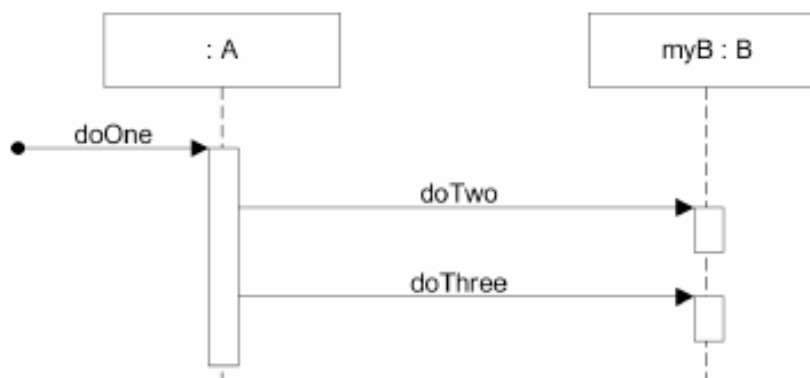
Sintassi	Nome	Semantica
	Messaggio sincrono	Il mittente aspetta che il destinatario ritorni dall'esecuzione del messaggio
	Messaggio asincrono	Il mittente invia il messaggio e continua l'esecuzione: non aspetta un Ritorno dal destinatario
	Messaggio di ritorno	Il destinatario di un messaggio precedente restituisce il focus di Controllo al mittente di quel messaggio
 	Creazione dell'oggetto Distruzione dell'oggetto	Il mittente crea un'istanza del classificatore specificato dal destinatario Il mittente distrugge il destinatario. Se la sua linea di vita ha una coda, questa termina con una X
	Messaggio trovato	il messaggio viene inviato dal di fuori dell'ambito dell'interazione. Si vuole mostrare la ricezione del messaggio ma non la provenienza.
	Messaggio perso	Il messaggio non raggiunge mai la sua destinazione. Si può usare per indicare condizioni di errore in cui i messaggi vanno persi.

6.2 Diagrammi di Interazione

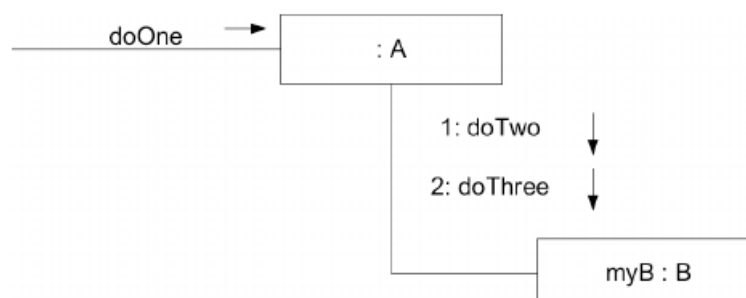
Il termine diagramma di interazione è una generalizzazione di 2 tipi più specifici di diagrammi di UML

- **Diagrammi di Sequenza**

Mostrano le interazioni in una specie di formato a steccato, in cui gli oggetti che partecipano all'interazione sono mostrati in alto, uno a fianco all'altro



- **Diagrammi di comunicazione** Mostrano le interazioni tra gli oggetti in un formato a grafo o rete, in cui gli oggetti possono essere posizionati ovunque nel diagramma



6.2.1 Vantaggi e Svantaggi

Tipo	Punti di forza	Punti di debolezza
Diagramma di sequenza	Mostra chiaramente la sequenza o l'ordinamento temporale dei messaggi. Opzioni di notazione numerose e dettagliate	Costringe a estendersi verso destra quando si aggiungono nuovi oggetti Consuma lo spazio orizzontale
Diagramma di comunicazione	Economizza lo spazio flessibilità nell'aggiunta di nuovi oggetti nelle 2 dimensioni	Più difficile vedere la sequenza dei messaggi. Meno opzioni di notazione.

Capitolo 7

Diagramma delle classi di progetto

7.1 Cosa è il Modello delle classi di progetto?

Il diagramma delle classi di progetto (DCD - Design Class Diagram) è un modello utilizzato per la modellazione statica degli oggetti, e serve a illustrare le classi, le interfacce e le relative associazioni adottando uno specifico punto di vista software

7.1.1 Differenza col Modello di Dominio

A differenza del **Modello di Dominio**, che rappresenta gli elementi da un punto di vista concettuale e slegato dal codice, il **Modello delle Classi di Progetto** definisce come il sistema sarà realmente implementato

7.2 Parole Chiave

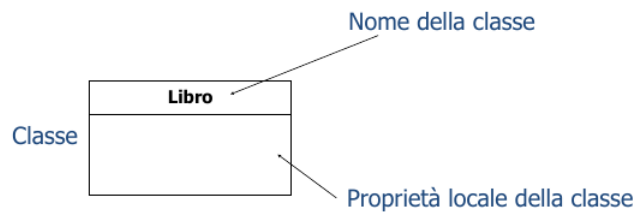
Parola Chiave	Significato	Esempio di suo
«actor»	il classificatore è un attore	Nei diagrammi delle classi, sopra al nome di un classificatore
«interface»	Il classificatore è un'interfaccia	Nei diagrammi delle classi, sopra al nome di un classificatore
{abstract}	l'elemento è astratto, non può essere istanziato	Nei diagrammi delle classi, dopo il nome di un classificatore o il nome di un'operazione
{ordered}	Un insieme di oggetti ha un ordine predefinito	Nei diagrammi delle classi, a un'estremità di associazione

7.3 Componenti del DCD

7.3.1 Classificato (Classi e interfacce)

Classi

Rappresentano la struttura portante e modellano un insieme di oggetti con attributi e operazioni comuni



7.3.2 Interfacce

Definiscono un "contratto" (un insieme di operazioni pubbliche senza implementazione) che le classi devono rispettare.

7.3.3 Attributi

Gli attributi modellano le proprietà locali e interne di una classe

In un DCD, a differenza del modello di dominio concettuale, presentano dettagli tecnici specifici

Sintassi

Includono il nome, il tipo di dato e l'eventuale molteplicità o valore iniziale

Visibilità

Simbolo	Nome	Semantica
+	Pubblico	Visibile a tutti
-	Privato	Visibile solo alla classe
#	Protetta	Visibile alla classe e ai suoi "discendenti"
~	Package	Visibile ad ogni elemento dello stesso package

7.3.4 Operazioni e Metodi

Operazioni

Sono le dichiarazioni della firma

Metodi

Sono l'implementazione pratica dell'operazione

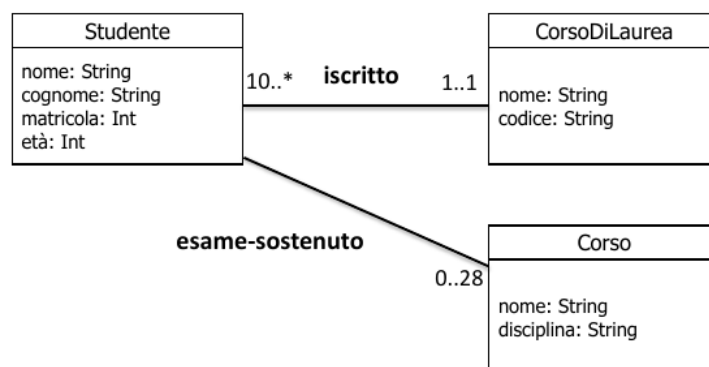
7.3.5 Associazioni

Le associazioni vengono disegnate come linee che uniscono le classi e si definiscono attraverso diverse proprietà

Nome e verso

Si può assegnare un nome descrittivo all'associazione.

Talvolta si accompagna a un fraccia (il verso) che aiuta solo a leggere l'associazione nel senso corretto



Ruoli

Si indicano scrivendo un nome all'estremità della linea per specificare quale ruola gioca quella classe nell'associazione.



Molteplicità

Sono dei limiti numerici posti vicino alla destinazione che indicano quanti oggetti di quella classe possono partecipare alla relazione in un dato istante



7.3.6 Associazioni n-arie